

Summary

This briefing explain the core principles of **Serializability** and **Concurrency Control** in **Database Management Systems (DBMS)**. The fundamental challenge in modern databases is to allow **concurrent transaction execution** for performance while preserving **data consistency**.

Pure **serial execution** guarantees correctness but is inefficient. **Non-serial (interleaved) execution** maximizes throughput but risks inconsistency. **Serializability** bridges this gap by ensuring that interleaved schedules produce results equivalent to some serial execution order.

This document explains:

- The role of schedules and isolation
- Conflict Serializability using **precedence graphs**
- View Serializability using **state-based validation**
- Strategic insights for exam-oriented and system-level understanding

1. The Foundation of Transaction Control

Transactions in a DBMS must satisfy the **ACID properties**, with **Isolation** being the central concern for concurrency control. Isolation ensures that even when transactions execute concurrently, each behaves as if it were running alone.

1.1 Understanding Schedules

A **Schedule** defines the order in which operations from multiple transactions are executed.

Serial Schedule

- One transaction completes fully before the next begins
- Always consistent
- Poor performance due to lack of concurrency

Non-Serial Schedule

- Operations of multiple transactions are interleaved
- High throughput and better resource utilization
- Can lead to inconsistency if not controlled

1.2 The Role of Serializability

Serializability ensures that a non-serial schedule produces the **same final database state** as some serial schedule.

It:

- Preserves correctness
- Retains the performance benefits of concurrency
- Acts as the formal correctness criterion for concurrent execution

2. Conflict Serializability

Conflict Serializability evaluates correctness by analyzing **conflicting operations** between transactions.

2.1 Conflict Operations

Two operations are said to **conflict** if all the following are true:

1. They belong to different transactions
2. They operate on the same data item
3. At least one operation is a Write (W)

Conflict vs. Non-Conflict Classification

- Read–Write (R–W): Conflict
- Write–Read (W–R): Conflict
- Write–Write (W–W): Conflict
- Read–Read (R–R): No Conflict
- Operations on different data items: No Conflict

2.2 The Precedence Graph Method

To test Conflict Serializability, a **Precedence Graph** (also called a Serialization Graph) is constructed.

Graph Construction

- **Vertices:** Each transaction ($T_1, T_2, T_3, \dots$)
- **Edges:** Directed edge from T_i to T_j if a conflicting operation of T_i occurs before T_j

Consistency Rule (CNC Rule)

- If the graph contains a **cycle** → Not Conflict Serializable
- If the graph is **acyclic** → Conflict Serializable (hence consistent)

Conflict Serializability is computationally efficient and is therefore the **first test applied** in practice.

3. View Serializability

View Serializability is a **weaker but more general** correctness criterion than Conflict Serializability.

3.1 Logical Relationship

- Every Conflict Serializable schedule is View Serializable
- A schedule that is **not Conflict Serializable** may still be **View Serializable**

Thus, View Serializability captures some correct schedules that Conflict Serializability rejects.

3.2 The Three-Step Verification Process

Two schedules (a non-serial schedule S_1 and a serial schedule S_2) are **View Equivalent** if all three conditions hold for every data item.

1. Initial Read Rule

The transaction that reads the initial value of a data item in S_1 must also read the initial value in S_2 .

2. Final Write Rule

The transaction that performs the last write on a data item in S_1 must also perform the last write in S_2 .

3. Read-from Rule (Intermediate / Dirty Read)

If a transaction T_i reads a value written by T_j in S_1 , then T_i must read the value written by T_j in S_2 .

If all three conditions are satisfied, the schedule is **View Serializable**, even if conflicts exist.

4. Key Strategic Insights

Performance vs. Consistency

- Non-serial schedules are essential for modern high-performance databases
- Serializability ensures that speed does not compromise correctness

Architectural Analogy

- Serializability can be viewed as the “older brother” of non-serial execution
- It inherits speed from interleaving and enforces discipline through consistency rules

Testing Hierarchy

1. **Conflict Serializability**
 - Faster, graph-based
 - First line of validation
2. **View Serializability**
 - More powerful but computationally complex
 - Used when conflict tests fail